REARRANGEABLE PANELS

functional specification

`IdleWorlds Fantasy Skin → IdleWorlds core. Frozen 2026-09.`

WHAT THIS IS

A complete, self-contained description of a shipped feature: the player drags panels into the order they want and the layout is remembered. It was built inside a browser extension that may not move a single node the app rendered, so it reorders with CSS only. You can move nodes. That makes about half of this document unnecessary for you — and the half that remains is the half that took the time.

HOW TO READ IT

Pseudo-code, not source. Sections marked DROP exist only because the extension could not touch the component tree. Sections marked KEEP are product decisions and measurements that survive any implementation.

```
READING ORDER IF YOU ARE SHORT ON BUDGET
1. §1 BEHAVIOUR        — what it does.
2. §3 PERSISTENCE      — the schema, and why it is per-layout.
3. §6 CROSS-COLUMN     — the measurement that decides your whole layout
                         approach. Read this before designing anything.
4. §9 MEASURED VALUES  — the table.
```

§1 BEHAVIOUR

The player enters an explicit REARRANGE MODE from a control in the main nav. In that mode every panel gains a grab surface. They drag a panel and the other panels reflow live around it; they drop it and the arrangement saves. Arrow keys do the same thing from the keyboard. Escape abandons a drag in progress. A Reset control restores the app's own order.

Outside the mode nothing is added to the page and no pointer handler exists.

Four invariants:

I1. The layout is remembered per player, indefinitely. I2. A panel the system cannot NAME is not draggable at all. See §4. An affordance that appears to work and then silently reverts is worse than no affordance. I3. The drag preview is exactly what will settle. Never an approximation. I4. The keyboard path is not a fallback. See §7.

§2 WHY A MODE, AND NOT ALWAYS-ON DRAGGING

KEEP the decision; the reasoning transfers.

This is an idle game. The player clicks inside these panels constantly, for hours. An always-live grab affordance on a panel — or worse, a draggable header row — turns every slightly-long click into a layout change.

The header row was also already full. It carries the app's own controls at one end and the collapse toggle at the other, with 8px between them. There was no third place to put a grip that was not a mis-click waiting to happen.

So: an explicit mode, and inside it the grab surface is the WHOLE PANEL. That removes the aiming problem entirely — there is nothing to miss — and it shields the app's own controls instead of competing with them.

ONE UX CONSEQUENCE, AND IT WAS REPORTED: a player who has not found the mode drags a panel and nothing happens at all, with no feedback explaining why. If you keep the mode, make entering it discoverable from the panels themselves, not only from the nav. If you drop the mode, use a press-and-hold delay rather than an immediate drag, for the reason in the first paragraph.

§3 PERSISTENCE

```
const StoredState = {
  key: 'panel-order',
  shape: {
    v: 1,                                  // version it from day one
    scopes: {
      // scope -> ordered list of panel ids
      '<route>|<layout>|<containerIndex>': ['panelA', 'panelB', '…'],
    },
  },
};
```

KEEP — the scope is three-part, and all three parts are load-bearing.

ROUTE. Arrangements are per page. The dashboard's order says nothing about the market's.

LAYOUT. This is the one that is easy to miss. The wide layout is TWO columns of four and three panels; the narrow layout is ONE column of all seven. A wide arrangement is not expressible as a narrow one. Storing them in one list means every resize past the breakpoint shuffles the player's panels. Store them separately and let the player arrange each.

CONTAINER. Which column, within the layout. Derived from painted position — left to right, then top to bottom — with source order as a tie-break so it is stable and repeatable.

KEEP — unknown panels are appended, never dropped. A panel the app ships later is not in any stored list. It must keep its natural position rather than jumping to the front or disappearing. The rule: reorder only the panels named in the stored list, among the slots those panels already occupy; everything else holds its index. See §5.

KEEP — the in-flight read race, same as the collapse feature: a player who drags before the stored map arrives must not have it overwritten a second later. Track ids touched this session; skip them when the read lands.

§4 PANEL IDENTITY

DROP the derivation. KEEP the warning.

The extension had to infer an id per panel from the DOM. That produced a bug worth knowing about even though you will not hit it the same way: the SAME PANEL resolved to two different ids depending on when you asked, because one source of the id only appeared after other code had run. One panel then owned two storage slots and its position depended on load timing.

For collapse this was survivable; a forgotten panel just stays open. For ORDERING it is not: a panel with no id has no position, and two panels resolving to one id collide.

Give every panel a literal permanent string id at its definition site. Never derive it from the title, the index, or anything the app renders.

KEEP — I2, the consequence for panels that still have no id. During the extension's life three panels per capture could not be named. They are NOT given a grab surface, and they are visibly marked as pinned. This was found the hard way: they were draggable, the drag appeared to work, and the next render put them back, because the save had nothing to record them under.

---

§5 THE ARRANGEMENT FUNCTION

```
function arrange(panels, storedOrder) {
  // panels: the container's children in SOURCE order, each {id, movable}
  // storedOrder: the ids the player arranged, in their chosen order
  if (!storedOrder?.length) return panels;

  const rank = new Map(storedOrder.map((id, i) => [id, i]));

  // The slots occupied by panels the stored list actually names…
  const slots = [];
  const known = [];
  panels.forEach((panel, index) => {
    if (panel.movable && panel.id && rank.has(panel.id)) {
      slots.push(index);
      known.push(panel);
    }
  });

  // …are redistributed among THOSE SLOTS ONLY, in the player's order.
  known.sort((a, b) => rank.get(a.id) - rank.get(b.id));
  const out = [...panels];
  slots.forEach((slot, i) => { out[slot] = known[i]; });
  return out;
}
```

KEEP — this exact shape. It is what makes an unknown panel hold its place instead of being shuffled to an end. A new panel, a pinned panel, or a non-panel child sitting between two panels all keep their index.

KEEP — I3, and it is subtle. A free-form drag can produce a sequence this function would NOT settle to. Worked example: [A, X, B] with X unnamed. Drag B to the front and the raw sequence is [B, A, X] — but re-running `arrange` gives [B, X, A], because X holds index 1. The player sees one thing during the drag and a different thing a frame after dropping.

The fix is not to reconcile them afterwards. Run the DRAG PREVIEW through this same function on every pointer move: derive the candidate id list from the raw sequence, then render `arrange(panels, candidate)`.

The preview then IS the settled answer by construction and the two cannot diverge.

§6 CROSS-COLUMN MOVEMENT — read this before designing the layout

KEEP — this is the most valuable measurement in the document.

The wide dashboard is two independent columns. Players ask to drag a panel from one to the other. The extension could not do it, because the CSS property it reorders with only reorders siblings. But the real obstacle is not the mechanism — YOU will hit it too, and it is a layout problem.

The obvious approach is one grid with two columns and panels placed into it. In a CSS grid, ROW TRACKS SPAN THE WHOLE ROW. A tall panel in column 1 stretches the row that a short panel in column 2 sits in, leaving a hole.

Measured on a real account's panel heights:

```
column 1 : 1031, 202, 35, 553   (Skill Actions, Current Action,
                                 Action Log collapsed, World Chat)
column 2 : 35, 35, 1039         (Inventory, Quests, World Bosses)
```

```
two independent columns → page height  1,821px
one shared two-column grid → page height 2,825px
dead whitespace introduced             1,004px
```

A 1,031px panel beside a 35px collapsed one stretches that row to 1,031px.

YOUR OPTIONS, in the order I would consider them:

A. Keep independent columns; allow reordering WITHIN a column, plus a single control that swaps the two columns wholesale. This is what shipped. Cheap, no holes, covers most of what players actually want.

B. Offer a single-column layout mode. In one column every panel is in one list and arbitrary ordering is real. This is the only option where "move it anywhere" is honestly true. Note the narrow layout ALREADY does this, so players below the breakpoint have it for free.

C. True two-dimensional placement with independent column heights. This needs either CSS masonry (check availability at your target) or a JS-measured column-balancing layout. It is a real project. Do not reach for it because cross-column drag "should be easy" — it is the layout that is hard, not the drag.

Do NOT dissolve the columns into a shared grid and hope the holes look acceptable. They were measured; they do not.

§7 THE KEYBOARD PATH — primary, not a fallback

KEEP if you reorder with CSS. ▒ Still worth keeping if you reorder the array, because it is the only way to use this feature without a mouse.

The grab surface is a real <button>. Focused, it responds to:

```
ArrowUp / ArrowLeft    move one position earlier
ArrowDown / ArrowRight move one position later
Home / End             move to either end
Escape                 abandon a pointer drag in progress
```

Steps skip over pinned and non-panel siblings, so a press never appears to do nothing.

After every move the panel's new position is announced through a polite live region: "Inventory moved to position 2 of 5." The accessible name of the grab surface carries the same information for anyone arriving by Tab.

WHY IT MATTERS MORE IN THE EXTENSION THAN IT WILL FOR YOU, and why you should still care: reordering with CSS leaves the DOM order — and therefore tab order and screen-reader reading order — untouched. Visual order and focus order then disagree, which is a genuine WCAG 2.4.3 failure. On this page the divergence is severe: with a long inventory there are roughly 1,200 focusable controls between two visually adjacent panels.

If you reorder the actual array, this problem disappears entirely. That is the single biggest correctness win available to you here, and it is a good reason to reorder the array rather than reach for the CSS property.

---

§8 THE DRAG

KEEP — the interaction rules.

THRESHOLD. A press must travel 5px before it becomes a drag, so a click on the grab surface stays a click.

POINTER CAPTURE. Capture the pointer on the grab surface at pointerdown. Events then retarget to it for the whole gesture, so the drag follows the pointer off the panel and outside the window with NO document-level listener, and nothing else on the page sees the events.

TOUCH. The grab surface must opt out of the browser's own touch gestures, or the browser claims the gesture for scrolling and the drag never starts on a phone — which is the single-column layout, where dragging is most useful. Having done that, you own scrolling during the drag: see autoscroll below.

AUTOSCROLL. Non-negotiable on this page. The dashboard column measures about 1,800px on a real account, so a drag from bottom to top cannot be completed inside one viewport. Scroll the page when the pointer is within 90px of the top or bottom edge, ramping to about 22px per frame at the very edge.

ESCAPE cancels and restores the order the drag started from.

KEEP — no ghost element follows the cursor. What moves is the LAYOUT: the other panels reflow live around the one being dragged, and the dragged panel gets a lifted treatment in place. This is better feedback than a ghost — the player sees the actual arrangement rather than a picture of one — and in the extension it was also the only affordable option, because repositioning an element every frame was measurably expensive. In your codebase the cost argument evaporates; the feedback argument does not.

KEEP — the lifted treatment is a filter and an opacity, NOT a transform. A transform creates a containing block for fixed-position descendants, which moves anything inside the card that is positioned that way.

```
function insertionIndexDuringDrag(pointerY, capturedGeometry) {
  // ▓ KEEP — and note WHY the geometry is captured once at drag start rather
  // than measured per frame.
  //
  // Measuring live rects each frame is a feedback loop in the literal sense:
  // the reflow this move causes changes the rects the next move reads, and the
  // panel oscillates between two slots. Panel heights do NOT change when their
  // order changes, so one measurement per panel at drag start stays true for
  // the whole gesture.
  const { contentTop, others, gap } = capturedGeometry;  // `others` excludes the dragged panel
  let edge = contentTop;
  let index = 0;
  for (const other of others) {
    if (pointerY <= edge + other.height / 2) break;
    index += 1;
    edge += other.height + gap;
  }
  return index;
}
```

DROP — everything about how the reorder is APPLIED. The extension wrote an attribute per panel that a static stylesheet turned into a CSS order value, specifically because that attribute was invisible to the machinery watching the page for changes. A whole drag cost zero observed mutations. None of that reasoning applies to you: reorder the array.

§9 MEASURED VALUES

```
const MEASURED = {
  dragThreshold:        '5px',
  autoscrollBand:       '90px from the viewport edge',
  autoscrollMaxSpeed:   '22px per frame',
  liftedTreatment:      'drop-shadow + 0.92 opacity (never a transform)',

  // The layout facts behind §6.
  wideLayout:           '2 independent columns, 4 + 3 panels',
  narrowLayout:         '1 column, all 7 panels',
  layoutBreakpoint:     '1280px',
  tallestPanel:         '1031px (Skill Actions, expanded)',
  shortestPanel:        '35px (any collapsed panel)',
  columnHeight:         '1821px',
  sharedGridHeight:     '2825px',
  sharedGridWhitespace: '1004px',   // the cost of one shared two-column grid
  longestColumn:        '~1800px',  // why autoscroll is required
};
```

§10 FAILURE MODES SEEN IN PRODUCTION

F1. A panel the player never touched hoisted itself to the top of a column. Cause: it was not given a position, so it kept the DEFAULT position value — which is a real position, equal to first place, and it won the tie on source order. Rule: give EVERY child of a reorderable container an explicit position, including ones the player cannot drag. A missing position is not neutral.

F2. Dragging a panel worked, then it snapped back a moment later. Cause: the panel had no id, so the save recorded nothing for it. See I2 and §4. It is now not draggable at all and is marked as pinned.

F3. The drag preview settled to a different order than it showed. Cause: §5, an unnamed panel between two named ones. Fix: run the preview through the same arrangement function as the commit.

F4. The player's saved layout was orphaned by resizing the window. Cause: the scope index was computed across containers that included hidden ones, and which containers are hidden changes at the breakpoint, so the same column owned different keys at different widths. Rule: derive the scope from what is actually laid out, and version the scope by layout (§3).

F5. "I click and drag a panel and nothing happens." Cause, twice: once a stale build, once the player had not entered the mode. See §2 — if you keep the mode, this WILL be reported, so make entering it visible from the panels rather than only from the nav.

---

§11 WHAT THIS COST, AND WHAT IT SHOULD COST YOU

In the extension: ~560 lines of JS, ~190 lines of CSS, ten opt-outs added to unrelated code elsewhere, and a static table of position rules because the positions could not be written inline.

In your codebase: an ordered array of panel ids in state, a persisted map keyed by route and layout, a drag handler, and the §7 keyboard handler. The array reorder gives you correct tab order, real cross-column movement inside a single-column layout, and animation for free.

The value here is §2 (why a mode), §3 (the scope), §5 (the arrangement rule and the preview/settle identity), §6 (the 1,004px measurement), §7 (why the keyboard path is not optional) and §10. Those are the parts that cost time. The mechanism is the easy half and yours is better.